

LAB 11

NAME: Hajra Sarwar

REG NO: 2023-BSE-022

CLASS: BSE-5A

SUBJECT: CC

LAB TITLE: GH CLI Codespaces + AWS + Terraform: Variables, Collections, Sensitivity & EC2 Provisioning

TASK:

Task 0: Lab Setup (Codespace & GH CLI)

1. Create a GitHub repo for the lab: CC_<YourName>_<YourRollNumber>/Lab11.
2. Open a **Codespace** (a Linux dev environment in the cloud) using GitHub CLI.

```
PS C:\Users\HP> gh codespace list
NAME                                DISPLAY NAME    REPOSITORY                                BRANCH  STATE    CREATED AT
psychic-tribble-wrq756xxww9x2964g  psychic tribble hajrasarwar11/skill-Bridge              main*   Shutdown about 12 days ago
fluffy-tribble-wrq756x47vr25q6q     fluffy tribble  hajrasarwar11/CC-HajraSarwar-022        main    Shutdown about 4 hours ago
PS C:\Users\HP>
```

3. SSH into the Codespace to work in its shell.

```
PS C:\Users\HP> gh codespace ssh -c fluffy-tribble-wrq756x47vr25q6q
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.
```

Task 1: Provider & Basic Variable (Variable Precedence)

Steps:

1. Create main.tf.
2. Edit main.tf and add provider:

```
lab_11 > main.tf
1 provider "aws" {
2   shared_config_files    = ["~/.aws/config"]
3   shared_credentials_files = ["~/.aws/credentials"]
}
```

3. Run terraform init to initialize Terraform in this directory.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform init
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

4. Define variable and Output:

```
main.tf U X
lab_11 > main.tf
6 variable "subnet_cidr_block" {
7   type = string
8 }
9
10 output "subnet_cidr_block_output" {
11   value = var.subnet_cidr_block
12 }
```

5. Run apply (first time without defaults):

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.1.0/24"
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

6. Add a default to the variable:

```
main.tf U X
lab_11 > main.tf
6 variable "subnet_cidr_block" {
7   type    = string
8   default = "10.0.0.0/24"
9 }
10
11 output "subnet_cidr_block_output" {
12   value = var.subnet_cidr_block
13 }
```

Run:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.0.0/24"
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

7. Export Environment Variable:

```
• @hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && export TF_VAR_subnet_cidr_block=10.0.20.0/24 && terraform apply -auto-approve

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.0.0/24" -> "10.0.20.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.20.0/24"
```

8. Create terraform.tfvars overriding values:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && cat terraform.tfvars && echo "---" && terraform apply -auto-approve
---

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.20.0/24" -> "10.0.30.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.30.0/24"
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

9. Override with -var:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve -var "subnet_cidr_block=10.0.40.0/24"

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.30.0/24" -> "10.0.40.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.40.0/24"
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

10. Show and unset env var:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && printenv | grep TF_VAR_ && echo "--- Unsetting TF_VAR_subnet_cidr_block ---" && unset TF_VAR_subnet_cidr_block && printenv | grep TF_VAR_ || echo "(No TF_VAR variables found after unset)"
TF_VAR_subnet_cidr_block=10.0.20.0/24
--- Unsetting TF_VAR_subnet_cidr_block ---
(No TF_VAR variables found after unset)
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

Task 2: Variable Validation & Sensitive / Ephemeral Variables

Steps:

1. Add **validation** to subnet_cidr_block to ensure CIDR format.

```
main.tf terraform.tfvars
lab_11 > main.tf
6 variable "subnet_cidr_block" {
7   type      = string
8   default   = ""
9   description = "CIDR block to assign to the application subnet"
10  sensitive  = false
11  nullable   = false
12  ephemeral  = false
13
14  validation {
15    condition     = can(regex("^[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}/[0-9]+$", var.sub
```

2. Test validation by giving invalid input → should fail.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve -var "subnet_cidr_block=10.0.0"

Error: Invalid value for variable

on main.tf line 6:
 6: variable "subnet_cidr_block" {
   |   var.subnet_cidr_block is "10.0.0"

The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24.

This was checked by the validation rule at main.tf:14,3-13.

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

3. Create a sensitive variable `api_session_token`:

```
lab_11 > main.tf
24 variable "api_session_token" {
25     type          = string
26     default       = ""
27     description   = "Short-lived API session token used during apply operations"
28     sensitive     = true
29     nullable      = false
30     ephemeral     = false
31
32     validation {
33         condition     = can(regex("^[A-Za-z0-9-]{20,}$", var.api_session_token))
34         error_message = "The API session token must be at least 20 characters and
```

4. Run with `-var` to observe sensitive output behavior:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve -var "api_session_token=my_API_session_Token"

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.30.0/24"

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

5. Check `terraform.state` for the sensitive output:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && cat terraform.tfstate | grep -A 10 "outputs"

"outputs": {
  "api_session_token_output": {
    "value": "my_API_session_Token",
    "type": "string",
    "sensitive": true
  },
  "subnet_cidr_block_output": {
    "value": "10.0.30.0/24",
    "type": "string"
```

6. Make variable ephemeral to hide from state:

```
lab_11 > main.tf
24 variable "api_session_token" {
29     nullable      = false
30     ephemeral     = true
```

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve -var "subnet_cidr_block=10.0.30.0/24" -var "api_session_token=my_API_session_Token"

Error: Ephemeral value not allowed

on main.tf line 39, in output "api_session_token_output":
39:   value      = var.api_session_token

This output value is not declared as returning an ephemeral value, so it cannot be set to a result derived from an ephemeral value.

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

- Set default to test local default:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.30.0/24"

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

Task 3: Project-level Variables, Locals, and Outputs

Steps:

- Add project/environment variables.

```
main.tf U X terraform.tfvars U

lab_11 > main.tf
43 variable "environment" {}
44
45 variable "project_name" {}
46
47 variable "primary_subnet_id" {}
48
49 variable "subnet_count" {}
```

- Use AWS CLI to get real subnet IDs.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && aws ec2 describe-subnets --filters "Name=availability-zone,Values=me-central-1a" --query "Subnets[].SubnetId" --output text
subnet-015aa793c89df0a82

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && cat terraform.tfvars
subnet_cidr_block = "10.0.30.0/24"

environment = "dev"
project_name = "lab_work"
primary_subnet_id = "subnet-015aa793c89df0a82"
subnet_count = 3
monitoring = true

@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

3. Define **locals** for derived values:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && cat locals.tf
locals {
  resource_name = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count      = var.subnet_count
  is_production      = var.environment == "prod"
  monitoring_enabled = var.monitoring || local.is_production
}
```

4. Add outputs to see local values after apply.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
is_production = false
monitoring_enabled = true
primary_public_subnet = "subnet-015aa793c89df0a82"
resource_name = "lab_work-dev"
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

Task 4: Maps and Objects

Steps:

1. Define a **map variable** tags → key-value pairs.

```
main.tf U X terraform.tfvars U locals.tf U
lab_11 > main.tf
73 variable "tags" {
74   type = map(string)
75 }
76
77 output "tags" {
78   value = var.tags
79 }
```

2. Apply Output showing tags:

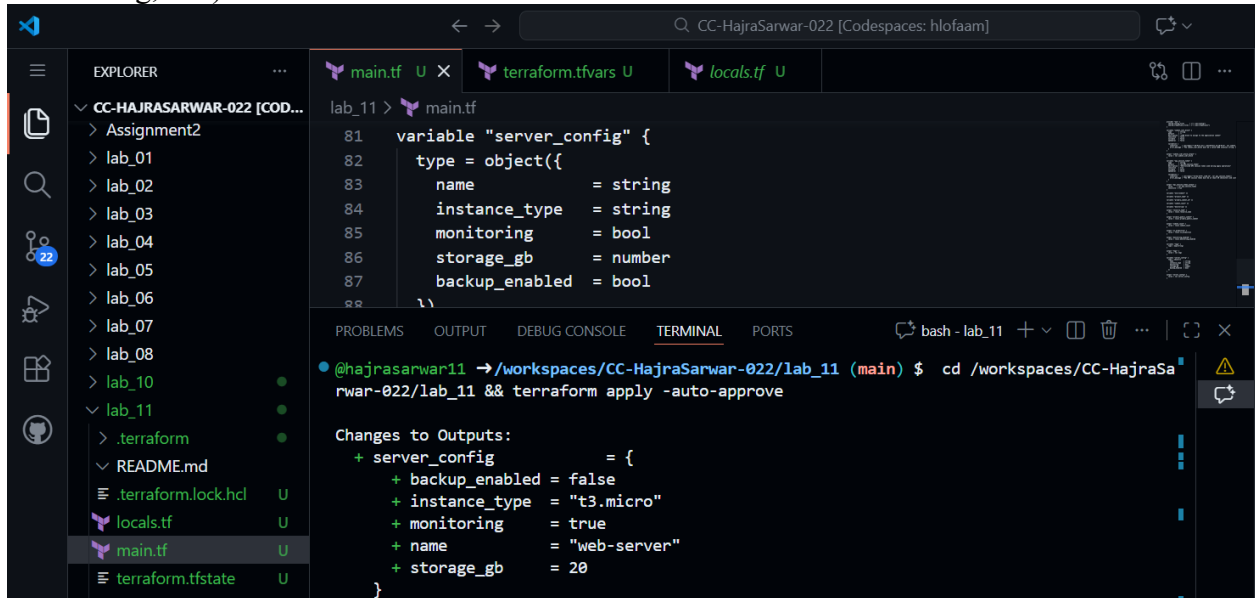
```
lab_11 > terraform.tfvars
9 tags = {
10   Environment = "dev"
11   Project     = "sample-app"
12   Owner       = "platform-team"
13 }
```

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
is_production = false
monitoring_enabled = true
primary_public_subnet = "subnet-015aa793c89df0a82"
resource_name = "lab_work-dev"
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner"       = "platform-team"
  "Project"     = "sample-app"
})
```

3. Define an **object variable** `server_config` → structured data for EC2 (name, instance_type, monitoring, etc.).



The screenshot shows the VS Code interface with the Explorer on the left showing a project structure for 'CC-HAJRASARWAR-022'. The main editor displays the `main.tf` file with a Terraform variable definition for `server_config`. The terminal at the bottom shows the command `terraform apply -auto-approve` being executed, and the output displays the values for the `server_config` output variable.

```
variable "server_config" {
  type = object({
    name           = string
    instance_type  = string
    monitoring      = bool
    storage_gb     = number
    backup_enabled = bool
  })
}
```

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve

Changes to Outputs:
+ server_config = {
+   backup_enabled = false
+   instance_type  = "t3.micro"
+   monitoring      = true
+   name           = "web-server"
+   storage_gb     = 20
}
```

Task 5: Collections: List, Tuple, Set & Mutation via Locals

Steps:

1. Define **list**, **tuple**, **set** variables in `main.tf`.



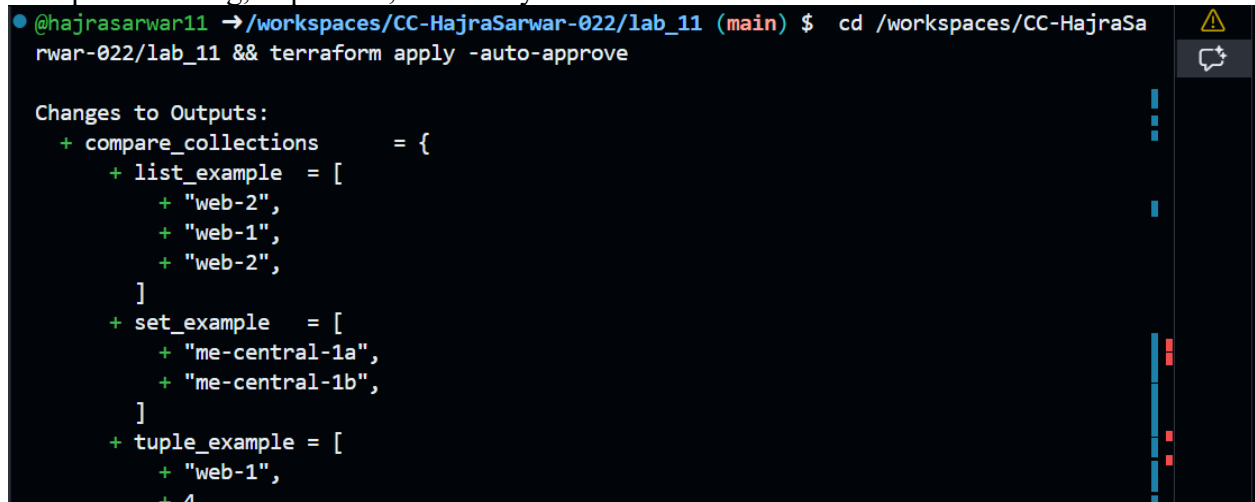
The screenshot shows the VS Code interface with the `main.tf` file open. It displays the definition of three variables: `server_names` (a list of strings), `server_metadata` (a tuple of string, number, and boolean), and `availability_zones` (a set of strings).

```
variable "server_names" {
  type = list(string)
  default = ["web-2", "web-1", "web-2"]
}

variable "server_metadata" {
  type = tuple([string, number, bool])
  default = ["web-1", 4, true]
}

variable "availability_zones" {
```

2. Compare ordering, duplicates, mutability:



The screenshot shows the terminal output of the `terraform apply` command. It displays the values for the `compare_collections` output variable, which includes a list example, a set example, and a tuple example.

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve

Changes to Outputs:
+ compare_collections = {
+   list_example = [
+     "web-2",
+     "web-1",
+     "web-2",
+   ]
+   set_example = [
+     "me-central-1a",
+     "me-central-1b",
+   ]
+   tuple_example = [
+     "web-1",
+     4,
+   ]
}
```


3. Demonstrate **mutations using locals** (setunion, etc.).

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && cat locals.tf
resource_name = "${var.project_name}-${var.environment}"
primary_public_subnet = var.primary_subnet_id
subnet_count      = var.subnet_count
is_production     = var.environment == "prod"
monitoring_enabled = var.monitoring || local.is_production

mutated_list = setunion(var.server_names, ["web-3"])
mutated_tuple = setunion(var.server_metadata, ["web-2"])
mutated_set = setunion(var.availability_zones, ["me-central-1c"])
}
```

4. Output comparisons to observe changes.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
Outputs:

api_session_token_output = <sensitive>
compare_collections = {
  "list_example" = tolist([
    "web-2",
    "web-1",
    "web-2",
  ])
  "set_example" = toset([
    "me-central-1a",
    "me-central-1b",
  ])
  "tuple_example" = [
    "web-1",
  ]
}
```

Task 6: Null, Any Type & Dynamic Values

Steps:

1. Null variable → optional value.

```
main.tf U X terraform.tfvars U locals.tf U
lab_11 > main.tf
81 variable "optional_tag" {
82     type      = string
83     description = "A tag that may or may not be provided"
84     default   = null
85 }
```

2. Merge null with locals → conditional inclusion.

```
main.tf U terraform.tfvars U locals.tf U X
lab_11 > locals.tf
1 locals {
12     server_tags = merge(
13         { Name = "web-server" },
14         var.optional_tag != null ? { Custom = var.optional_tag } : {}
15     )
}
```

3. Output:


```
lab_11 > main.tf
81   output "optional_tag" {
82     value = local.server_tags
83   }
84

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
Outputs:

api_session_token_output = <sensitive>
compare_collections = {
  "list_example" = tolist([
    "web-2",
    "web-1",
    "web-2",
  ])
  "set_example" = toset([
    "me-central-1a",
    "me-central-1b",
  ])
}
```

4. Apply Output after setting optional tag:

```
lab_11 > terraform.tfvars
23   optional_tag = "dev"
24

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
Outputs:

api_session_token_output = <sensitive>
compare_collections = {
  "list_example" = tolist([
    "web-2",
    "web-1",
    "web-2",
  ])
  "set_example" = toset([
    "me-central-1a",
    "me-central-1b",
  ])
  "tuple_example" = [
```

5. Test dynamic variable with different types:

(a). String:

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = "hello"
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

(b). Number:

```
@hajrasarwar11 → /workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = 42
```

(c). List:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
"Environment" = "dev"
"Owner" = "platform-team"
"Project" = "sample-app"
})
value_received = [
  "a",
  "b",
  "c",
]
```

(d). Map / Object:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cd /workspaces/CC-HajraSarwar-022/lab_11 && terraform apply -auto-approve
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = {
  "cpu" = 4
  "name" = "server"
}
```

(e). Null:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve
"storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
```

Task 7: Git Ignore

Steps:

- Create .gitignore and add entries to exclude sensitive files:

```
main.tf U  .gitignore U X  locals.tf U
.gitignore
1  .terraform/*
2  *.tfstate
3  *.tfstate.*
4  *.tfvars
5  *.pem
```

Task 8: Build Real Infrastructure (VPC, Subnet, IGW, Routing)

Steps:

1. Clean previous example values.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cat terraform.tfvars
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cat locals.tf
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cat main.tf
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
```

2. Define Variables in main.tf:

```
main.tf U X
lab_11 > main.tf
6 variable "vpc_cidr_block" {}
7 variable "subnet_cidr_block" {}
8 variable "availability_zone" {}
9 variable "env_prefix" {}
```

3. Create VPC in main.tf and Add VPC resources to main.tf:

```
main.tf U X
lab_11 > main.tf
11 resource "aws_vpc" "myapp_vpc" {
12     cidr_block = var.vpc_cidr_block
13     tags = {
14         Name = "${var.env_prefix}-vpc"
15     }
16 }
```

4. Create Subnet in the VPC Add the subnet resource to main.tf:

```
main.tf U X
lab_11 > main.tf
18 resource "aws_subnet" "myapp_subnet_1" {
19     vpc_id          = aws_vpc.myapp_vpc.id
20     cidr_block       = var.subnet_cidr_block
21     availability_zone = var.availability_zone
22     tags = {
23         Name = "${var.env_prefix}-subnet-1"
24     }
```

5. Populate terraform.tfvars In terraform.tfvars add:

```
main.tf U terraform.tfvars X
lab_11 > terraform.tfvars
1 vpc_cidr_block      = "10.0.0.0/16"
2 subnet_cidr_block   = "10.0.10.0/24"
3 availability_zone    = "me-central-1a"
4 env_prefix          = "dev"
```

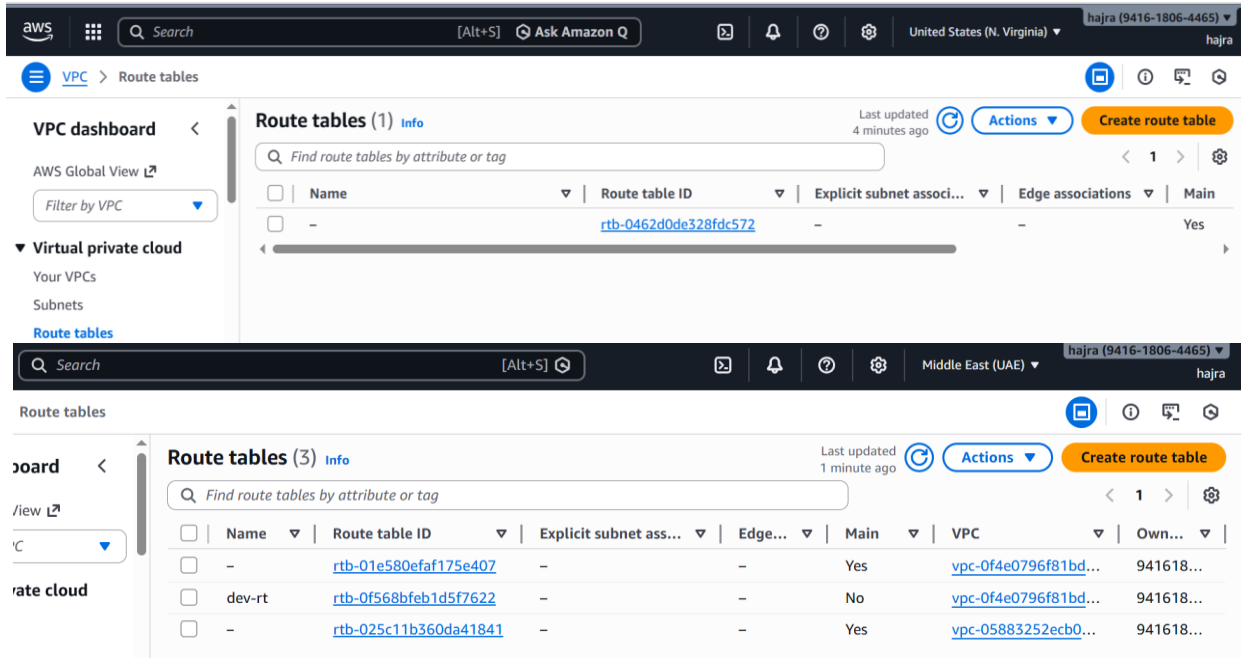
6. Apply to create VPC and Subnet Initialize (if needed) and apply:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve
aws_subnet.myapp_subnet_1: Creating...
aws_subnet.myapp_subnet_1: Creation complete after 1s [id=subnet-04a236d6014424ecf]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

Name	Subnet ID	State	VPC	Block	IPv4 CIDR
dev-subn...	subnet-04a236d6014424ecf	Available..	vpc-0f4e0796f81bdc432 d...	Off	10.0.10.0/24
-	subnet-015aa793c89df0a82	Available..	vpc-05883252ecb09ac3e	Off	172.31.32.0/20
-	subnet-084c39d400b49a9b4	Available..	vpc-05883252ecb09ac3e	Off	172.31.16.0/20
-	subnet-0336f1ba99f13aae0	Available..	vpc-05883252ecb09ac3e	Off	172.31.0.0/20

7. Create Internet Gateway and Route Table (custom) Add the Internet Gateway and a custom Route Table to main.tf:



The screenshot shows the AWS Management Console interface for Route Tables. The top section is for the 'United States (N. Virginia)' region, showing one route table with ID 'rtb-0462d0de328fd572'. The bottom section is for the 'Middle East (UAE)' region, showing three route tables: 'rtb-01e580efaf175e407', 'rtb-0f568bfeb1d5f7622', and 'rtb-025c11b360da41841'. The table columns include Name, Route table ID, Explicit subnet association, Edge associations, Main, and VPC.

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
-	rtb-0462d0de328fd572	-	-	Yes	

Name	Route table ID	Explicit subnet associ...	Edge...	Main	VPC	Own...
-	rtb-01e580efaf175e407	-	-	Yes	vpc-0f4e0796f81bd...	941618...
dev-rt	rtb-0f568bfeb1d5f7622	-	-	No	vpc-0f4e0796f81bd...	941618...
-	rtb-025c11b360da41841	-	-	Yes	vpc-05883252ecb0...	941618...

8. Associate the Route Table with the Subnet Add the association resource to main.tf:

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve
+ id          = (known after apply)
+ region      = "me-central-1"
+ route_table_id = "rtb-0f568bfeb1d5f7622"
+ subnet_id    = "subnet-04a236d6014424ecf"
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_route_table_association.a_rt_b_subnet: Creating...
aws_route_table_association.a_rt_b_subnet: Creation complete after 0s [id=rtbassoc-007f90ac2f09253f9]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

9. Switch to default route table (use VPC default route table) Now remove (or comment out) the custom route table and association resources from main.tf:

```
lab_11 > main.tf
35 resource "aws_default_route_table" "main_rt" {
36     default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
37
38     route {
39         cidr_block = "0.0.0.0/0"

```

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve
aws_route_table.myapp_route_table: Destruction complete after 0s
aws_default_route_table.main_rt: Creation complete after 2s [id=rtb-01e580efaf175e407]

Apply complete! Resources: 1 added, 0 changed, 2 destroyed.
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

Task 9: Security Group, Key Pair, EC2, user_data & nginx

Steps:

1. Add variables to main.tf :

```
main.tf U X terraform.tfvars
lab_11 > main.tf
6 variable "vpc_cidr_block" {}
7 variable "subnet_cidr_block" {}
8 variable "availability_zone" {}
9 variable "env_prefix" {}
10
11 variable "my_ip" {}
```

2. Get your public IP and set terraform.tfvars From the Codespace shell:

```
main.tf U terraform.tfvars X
lab_11 > terraform.tfvars
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 availability_zone = "me-central-1a"
4 env_prefix = "dev"
5 my_ip = "4.240.39.192/32"
6 instance_type = "t3.micro"
```

3. Create the Security Group (main.tf) Add this resource to main.tf:

```
main.tf U X terraform.tfvars
lab_11 > main.tf
1 resource "aws_default_security_group" "myapp_sg" {
2   vpc_id = aws_vpc.myapp_vpc.id
3
4   ingress {
5     from_port = 22
6     to_port = 22
7     protocol = "tcp"
8     cidr_blocks = [var.my_ip]
9   }
10 }
```

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-approve
in file "terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

Q Search [Alt+S]

Middle East (UAE) hajra (9416-1806-4465) hajra

Security Groups (2) Info

Find security groups by attribute or tag

	Name	Security group ID	Security group name	VPC ID	Description
☐	-	sg-0567ae2c936d1bb1c	default	vpc-05883252ecb09ac3e	default VPC security group
☐	dev-sg	sg-03bf35e8fa634df80	default	vpc-0f4e0796f81bdc432	default VPC security group

4. Create an AWS key pair and save locally Create a key pair and store the private key in your Codespace. Do NOT commit the .pem file.

```
main.tf U MyED25519Key.pem terraform.tfvars
MyED25519Key.pem
1 -----BEGIN OPENSSH PRIVATE KEY-----
2 b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAtz
3 c2gtZWQyNTUxOQAAACC5zF/ChelScsTpf9zFWsma16AIvdTelJpyGyzjUGnMagAA
4 AIiq9DhqqvQ4agAAAAtzc2gtZWQyNTUxOQAAACC5zF/ChelScsTpf9zFWsma16AI
5 vdTelJpyGyzjUGnMagAAAEAwUBATAFBgMrZXAEIgQgRJdY19GbA3INbUyL85CP
6 frnMX8KF6VJyx01/3MvayZrXoAi91N4uOnIbLONQacwCAAAAAECAwQF
7 -----END OPENSSH PRIVATE KEY-----
```

5. Launch EC2 instance using key pair and subnet.

```
main.tf U X terraform.tfvars
lab_11 > main.tf
1 resource "aws_instance" "myapp-server" {
2   ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023
3   instance_type = var.instance_type
4   subnet_id     = aws_subnet.myapp_subnet_1.id
5   security_groups = [aws_default_security_group.default_sg.id]
6   availability_zone = var.availability_zone
7   associate_public_ip_address = true
8   key_name       = "MyED25519Key"
}
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform apply -auto-app
rove
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-06ad7c72ba6ad3ce6]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
aws_instance_public_ip = "3.29.124.17"
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```

6. SSH into instance to verify connection.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ ssh -i MyED25519Key.pem ec2-user@3.29.124.17
The authenticity of host '3.29.124.17 (3.29.124.17)' can't be established.
ED25519 key fingerprint is SHA256:FoYeppI/wpOjQ20dNidYZrNkwCe7v0+pmdIJ6jWPKFc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.29.124.17' (ED25519) to the list of known hosts.

#_
~\ #### Amazon Linux 2023
~~ \#####\
~~ \###|
~~ \#/ https://aws.amazon.com/linux/amazon-linux-2023
~~ V~' '->
~~~
~~~_./_/_/
~~~/_/_'
~/m/'
[ec2-user@ip-10-0-10-191 ~]$
```

7. Generate SSH key locally, register with Terraform-managed key. ssh-keygen output and files in ~/.ssh:


```
[ec2-user@ip-10-0-10-191 ~]$ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /home/ec2-user/.ssh/id_ed25519
Your public key has been saved in /home/ec2-user/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:UNa2l8Bnd6WmFoLb9RAIr6GCr1sD1IbCATPQhMtgQZs ec2-user@ip-10-0-10-191.me-central-1.co
mpute.internal
The key's randomart image is:
+--[ED25519 256]--+
|OBo  ++ .. o|
|++=o  o o* o....|
|=Eo o . o.o+=oo.
|. + o  o =.oo*
| o . . S .. o .
| o .
| +
| o .
| o.
+----[SHA256]-----+
[ec2-user@ip-10-0-10-191 ~]$ 
[ec2-user@ip-10-0-10-191 ~]$ ls -l ~/.ssh
total 12
-rw-----. 1 ec2-user ec2-user 94 Jan 23 14:07 authorized_keys
-rw-----. 1 ec2-user ec2-user 452 Jan 23 14:17 id_ed25519
-rw-r--r--. 1 ec2-user ec2-user 135 Jan 23 14:17 id_ed25519.pub
[ec2-user@ip-10-0-10-191 ~]$
```

aws_key_pair resource and updated aws_instance key_name:

```
main.tf U X terraform.tfvars
lab_11 > main.tf
1 resource "aws_key_pair" "ssh_key" {
2   key_name = "serverkey"
3   public_key = file("~/ssh/id_ed25519.pub")
4 }
5 resource "aws_instance" "myapp-server" {
6   ami = "ami-05524d6658fcf35b6" # Amazon Linux 2023
7   instance_type = var.instance_type
8   subnet_id = aws_subnet.myapp_subnet_1.id
9   security_groups = [aws_default_security_group.myapp_sg.id]
10  availability_zone = var.availability_zone
11  associate_public_ip_address = true
12  key_name = aws_key_pair.ssh_key.key_name
```

Cleanup

Steps:

1. Destroy all Terraform-managed resources: `terraform destroy -auto-approve`.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ terraform destroy -auto-approve
aws_instance.myapp-server: Still destroying... [id=i-06ad7c72ba6ad3ce6, 01m20s elapsed]
aws_instance.myapp-server: Destruction complete after 1m21s
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-04a236d6014424ecf]
aws_default_security_group.myapp_sg: Destroying... [id=sg-03bf35e8fa634df80]
aws_default_security_group.myapp_sg: Destruction complete after 0s
aws_subnet.myapp_subnet_1: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0f4e0796f81bdc432]
aws_vpc.myapp_vpc: Destruction complete after 1s

Destroy complete! Resources: 6 destroyed.
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $
```


2. Verify state files are clean.

```
@hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cat terraform.tfstate
{
  "outputs": {},
  "resources": [],
  "check_results": null
}
● @hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ cat terraform.tfstate.backup
up
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 34,
  "lineage": "827dc526-61f1-5946-aa33-8af7387e53a4",
  "outputs": {
    "aws_instance_public_ip": {
      "value": "3.29.124.17",
      "type": "string"
    }
  },
  "resources": [
```

3. Ensure .pem and .tfstate files are not committed.

```
● @hajrasarwar11 →/workspaces/CC-HajraSarwar-022/lab_11 (main) $ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ../.devcontainer/devcontainer.json
        deleted:    ../UbuntuMachine/README.md
        deleted:    ../UbuntuMachine/start-desktop.sh
        deleted:    ../UbuntuMachine/stop-desktop.sh
```
